Distributed Systems

Events

def An event is an occurrence at a single process that causes a transition in its local state. It is formally represented as:

$$e_i^k = \text{K-th event of process } i$$

Relationship "Happened Before"

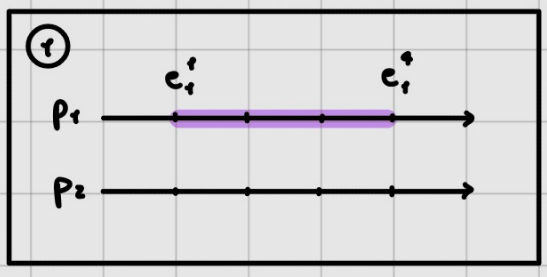
def The event e_i^l "happened before" e_j^m if:

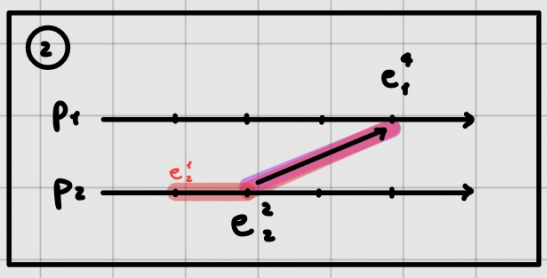
① $i = j$ and $l < m$ (same process and e^l happens before e^m)

OR

② e_i^l is the sender of the message received by e_j^m

is this def complete? Based on that $e_2^1 \not\longrightarrow e_1^4$





Local State

def The Local State of a process is simply the "snapshot" of the process in a specific situation. It is formally represented as:

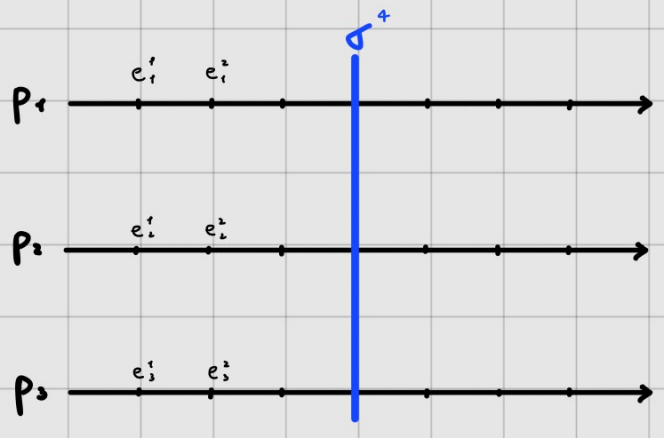
$$\sigma_i^k = \text{state of process } i \text{ after } k \text{ events } (e_i^1 \dots e_i^k)$$

Global State

def The Global State is a "snapshot" of the entire system. It is constructed from the union of the local state of every process. It is formally represented as:

$$\sigma^k = \text{state of all processes after } k \text{ events} = (\sigma_1^k, \sigma_2^k, \dots, \sigma_n^k)$$

N.B. A global state is actually a "straight cut" on the time



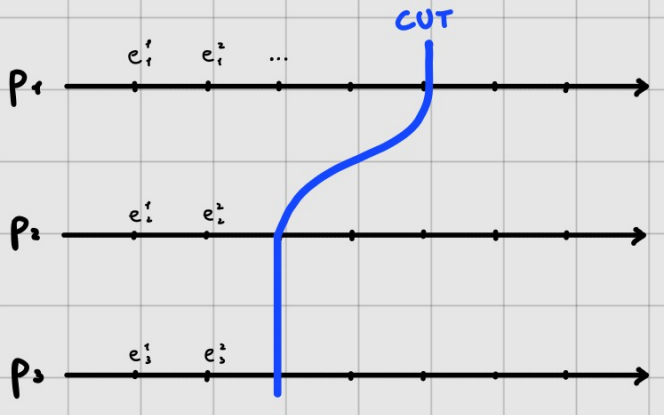
$$\sigma^4 = (\sigma_1^4, \sigma_2^4, \sigma_3^4)$$

Cut

def A "cut" is a set of events that contain the first k_i events of process i, for each process

prefix

same i ???



$$C = (\sigma_1^5, \sigma_2^4, \sigma_3^4)$$

However, we are not interested in "every cut". We only need "consistent cuts".

Consistent Cuts

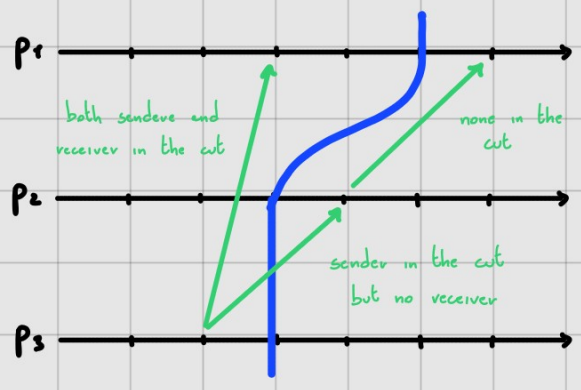
def A cut C is consistent iff: $e \longrightarrow e' \wedge e' \in C \Rightarrow e \in C$

(iff = if and only if; $\longrightarrow$ = happens before)

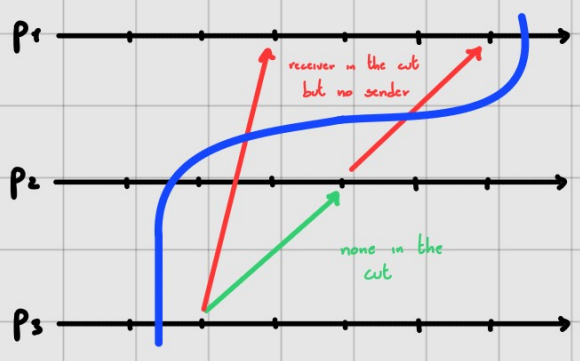
if e' receives a message from e, both e and e' must be in the cut

NON necessariamente il contrario. Se e manda un messaggio a e' ed $e \in C$, e' potrebbe anche NON appartenere a C

☑ CONSISTENT



☒ NOT CONSISTENT



Run

def A "run" is a sequence of events that represents a specific execution

As before, we can also define "what is a consistent run?"

def A run R is consistent iff: $e \longrightarrow e' \Rightarrow e$ appears before e' in R

(iff = if and only if; $\longrightarrow$ = happens before)

And, basically, a Run is a sequence of Cuts. So, we could cut a Run to get a Cut.

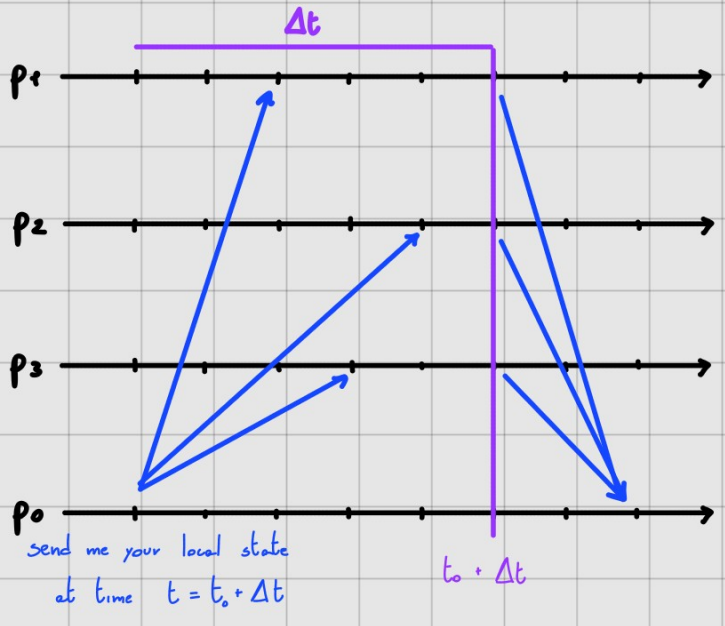
$$R = \overline{\overline{e_1^1 \; e_1^2 \; e_1^3}^{\,C'} \mid e_1^4 \; e_1^5} \mid e_1^6 \; \ldots \qquad (C'' \text{ over } C')$$

Lamport Clock

Let's start from an idealized world where capturing a consistent snapshot of the system's global state is very easy, thanks to 2 different assumptions:

- GLOBAL CLOCK: All processes share the exact same physical clock, meaning the clock drift is negligible or zero $(\Delta d = 0)$
- SYNCHRONOUS NETWORK: There is a know maximum Δt time needed to deliver messages.

Under these assumptions the following simple protocol works properly:



1) A coordinator process (p_0) broadcasts a request at time t_0, saying: "Send me your local state at tim $t = t_0 + \Delta t$"

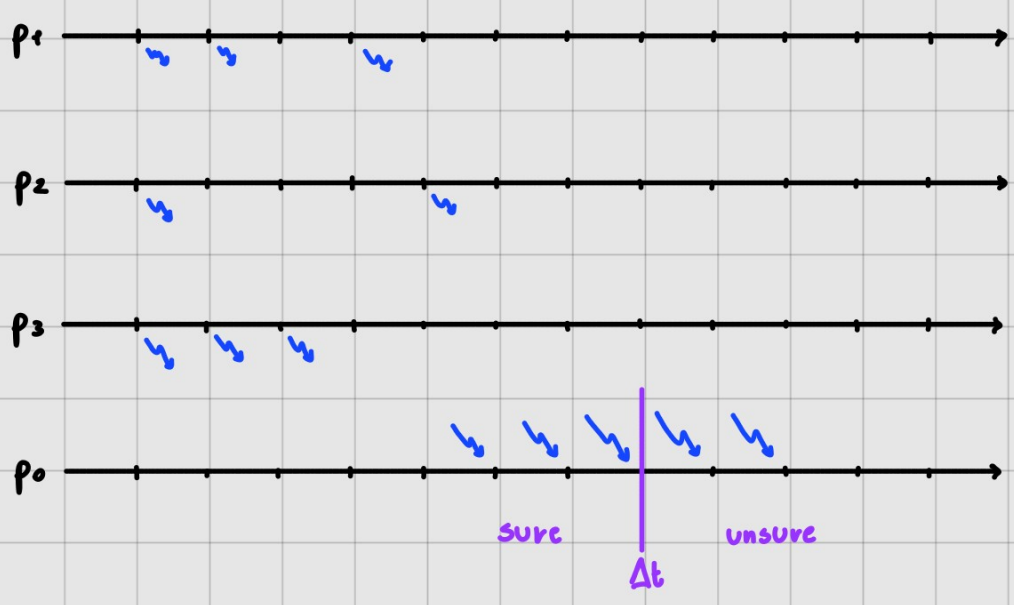
2) Because the network is synchronous all processes receives the request in time (before Δt time)

3) Thank to the global clock all processes record their local state at the exact same time

However, in real world distributed systems this protocol cannot actually work. Physical clocks inevitably drifts and network congestion, failures, retransmissions etc... make the message delays unpredictables. So, we need to change the protocol in order to eliminate these 2 assumption.

Removing the Global Clock

IDEA: Instead of asking processes to record their state at a specific time, every process sends a message to p_0 whenever local event happens. Then, p_0 will reconstruct the run to create the global snapshot.



NOTE: At this moment we are still relying on the SYNCHRONOUS NETWORK assumption. Because message delay is bounded by Δt p_0 knows that any event older than Δt has defently arrived.

Removing the Synchronous Assumption

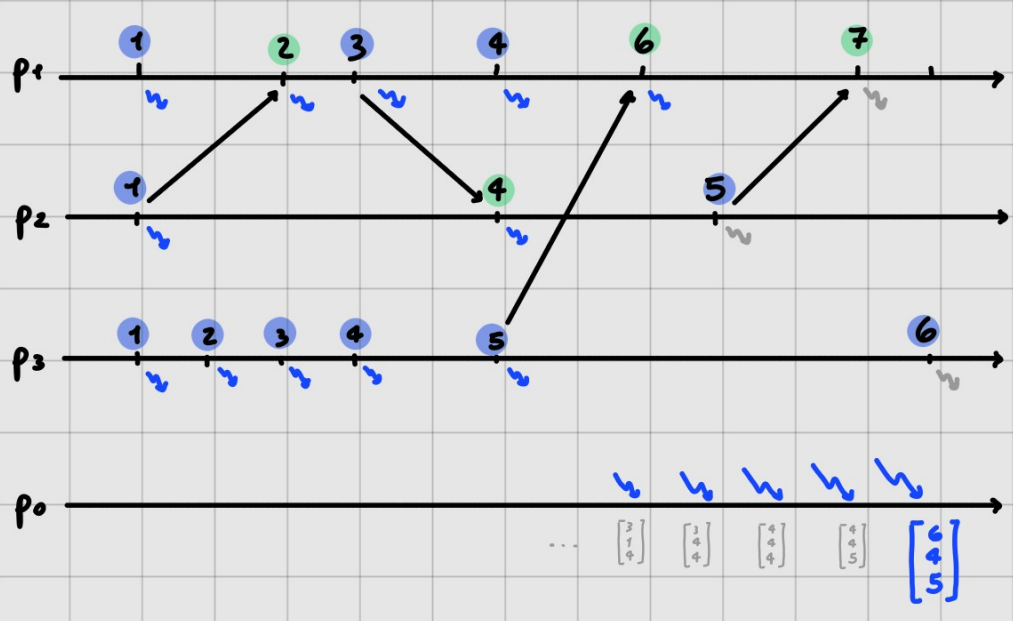
IDEA: To understand the system's global state we do not actually need to know the exact timestamp an event occurred. We only care about the order of the events. So, we can introduce the Clock Condition and the Lamport Clock to reorder messages to their "logical clock".

- (WEAK) CLOCK CONDITION: $e \xrightarrow{\text{happened before}} e' \Rightarrow T(e) < T(e')$ (Timestamp) (NOTE: $\Leftarrow$ it's not true)

- LAMPORTS CLOCK: Each process mantains an integer counter C and attaches it in every message.

  - At each LOCAL event increment $\Rightarrow C = C + 1$
  - Receiving a message with timestamp $t \Rightarrow C = \max(C, t) + 1$

Finally, to reconstruct the order of the events a node must ensure that no message with an older timestamp will arrive. To make this we can use a LAST MESSAGE VECTOR. Each process tracks the latest timestamp received from the others in a vector $[last_1, \dots, last_n]$. Events with timestamps up to $\min(last_1, \dots, last_n)$ can be safely committed because no future message can have smaller timestamp.



PROBLEM: If a process become idle its local counter C stops increasing. Consequently, its entry in the LAST MESSAGE VECTOR remains low freezing the global minimum. In that case the other processes can not commit their events because they are waiting on the silent process.

SOLUTION: Heartbeats / Periodic Null Message

(Strong) Clock Condition

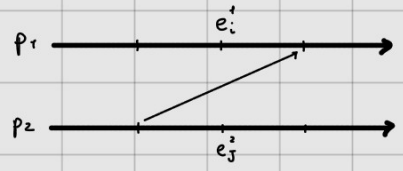
The previous (WEAK) CLOCK CONDITION "$e \longrightarrow e' \Rightarrow T(e) < T(e')$" could not say if e happened before e' or if they were simply concurrent systems. To solve this we want to achieve a STRONG CLOCK CONDITION:

$$e \longrightarrow e' \iff T_s(e) \leqslant T_s(e')$$

To achieve this bidirectional relationship we need to look at the entire history of an event.

Exercise: Is it possible that $e_i^l \not\longrightarrow e_j^m \wedge e_j \not\longrightarrow e_i$?

Yes!



Event's History

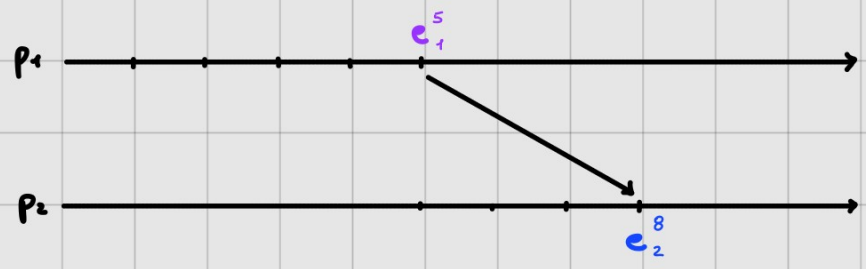
def The History of the event e is the set of events that happened before e
It is formally represented as:

$$H(e) = \{ e' \text{ t.c. } e' \longrightarrow e \}$$

(happened before)

NB if $e_i \longrightarrow e_j$ then $e_i \in H(e_j) \wedge H(e_i) \subseteq H(e_j)$

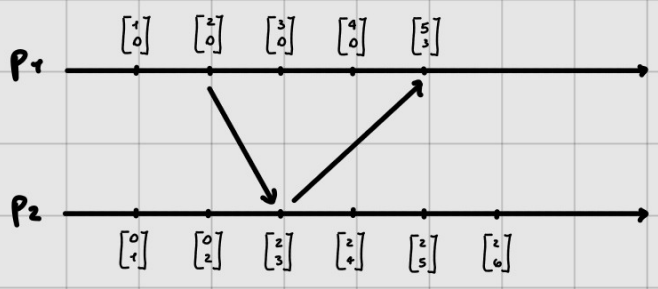
Vector Clock



Now we can define $T_S(e_2^8) = H(e_2^8) = \{e_1^5, e_2^8\}$ and write it "compressed" as a vector (known as Vector Clock) where each element v_i represents the latest know logical clock of the process p_i:

$$T_S(e_2^8) = H(e_2^8) = \begin{bmatrix} 5 \\ 8 \end{bmatrix} \begin{matrix} \leftarrow p_1 \\ \leftarrow p_2 \end{matrix}$$

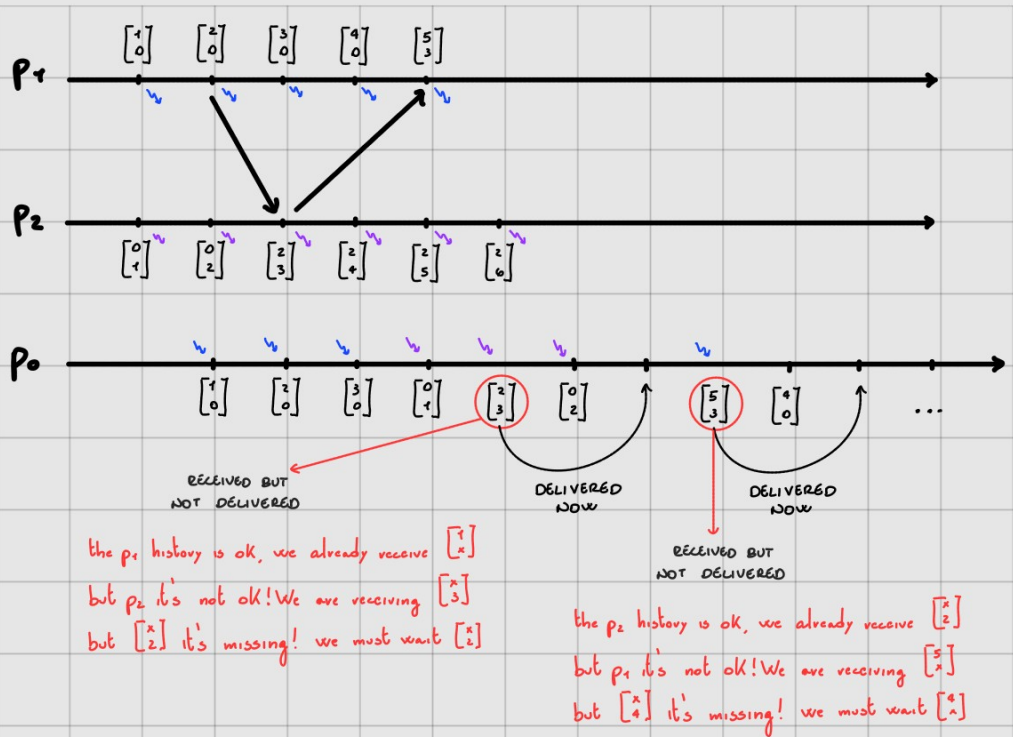
Exercise: Compute the History of each event



At this point we can also define that: "A vector is smaller than or equal to another iff every single component is smaller then or equal to its corrisponding component"
Formally:

$$\begin{bmatrix} x \\ y \end{bmatrix} \leqslant \begin{bmatrix} x' \\ y' \end{bmatrix} \iff x \leqslant x' \wedge y \leqslant y'$$

Because Vector Clock perfectly track all the order dependences, the coordinator p_0 will be able to detect if messages have arrived out of order (and potentially reorder them).



In this way p_0 can "reconstruct the Run" that, thanks to Vecto Clock, it's also CONSISTENT!

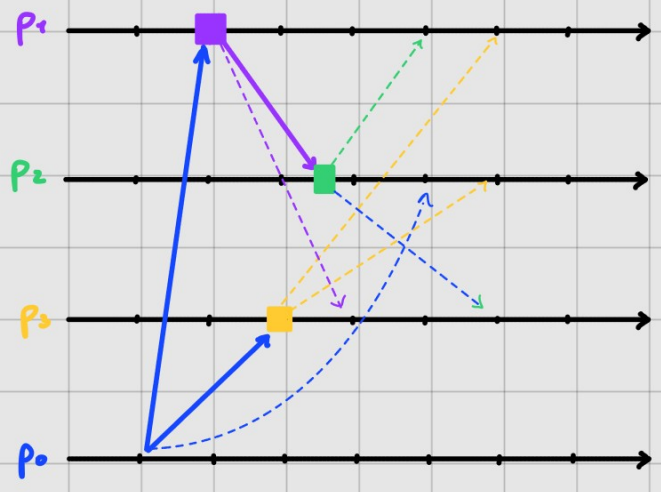
Chandy - Lamport Clock

The Chandy - Lamport Clock allows to take a picture of the global state without halting the underlying execution.

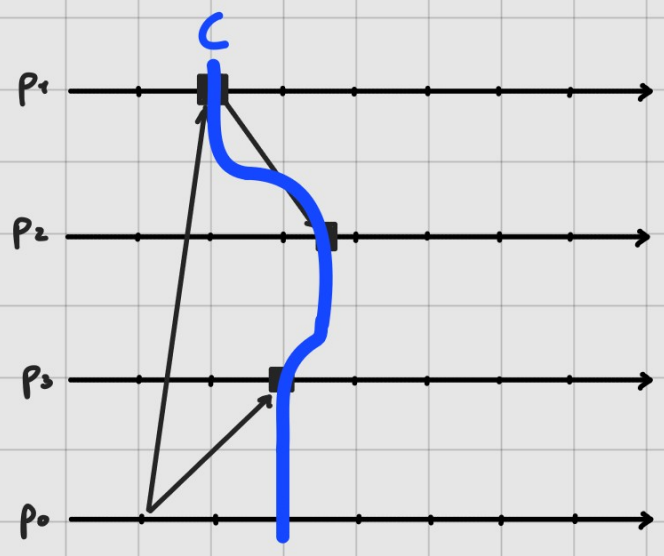
IDEA:

1) Each process sends the "take a snapshot" message to everybody else AS SOON AS it receives the message

   ↳ Example: p_2 sends the message as soon as it is received by p_1, and not from p_0



2) The cut on the exact moment each process take the snapshot forms the global state C.

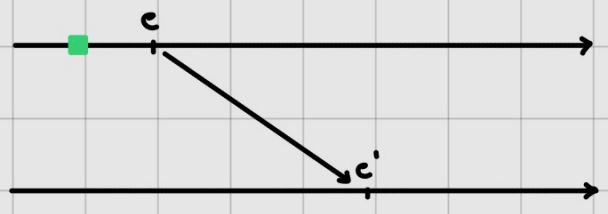


But: "is this cut C consistent?"

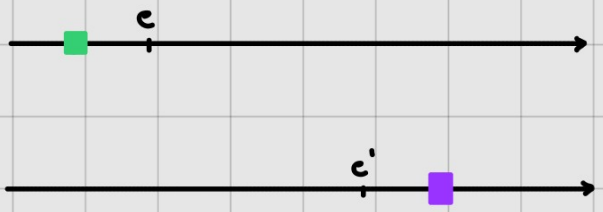
Dim. To prove that the Chandy-Lamport algorithm always generates a consistent cut we can use a proof by contradiction:

HP: $e \longrightarrow e' \wedge e' \in C \Rightarrow e \notin C$

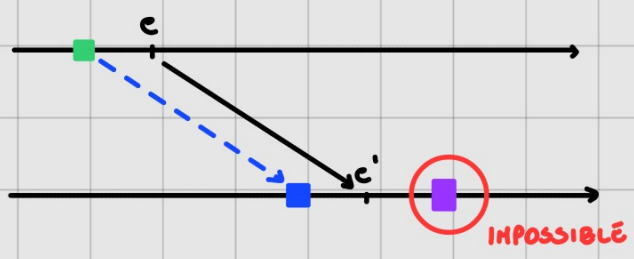
Since we are assuming that $e \notin C$, the cut C must have been taken BEFORE e.



On the contrary, since we know $e' \in C$, the cut C must have been taken AFTER e'



However, the Chandy-Lamport Clock strongly relies on the assumption that communication channels are FIFO. So, because the snapshot marker is sent as soon as possible, if the snapshot was taken BEFORE e the message was also sent before e and, since the channel is FIFO and $e \longrightarrow e'$, it must have arrived before e'.

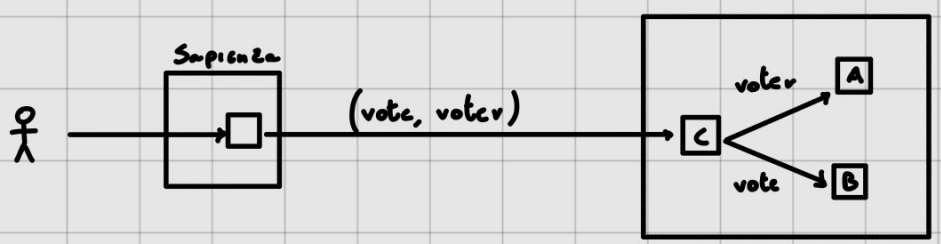


So, we have proved that: $e \longrightarrow e' \wedge e' \in C \Rightarrow e \in C$

So, the Chandy-Lamport Clock guarantees perfectly consistent cuts / global states.

Atomic Commit

Let's start from an example: The 2020 elections in Sapienza. During the COVID pandemic these elections were done remotelly. Consequently they needed a distributed system to collect the (vote, voter) pairs without storing the 2 information together (to ensure the vote anonimity).



(we need to know the voter to avoid that someone could vote twice)

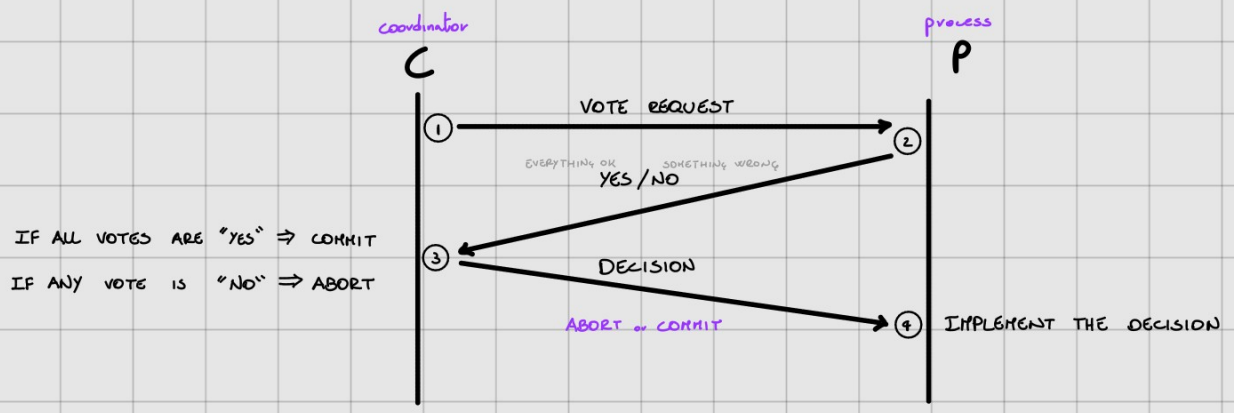
IDEA:

1) register the voter on A
2) register the vote on B
3) delete the pair (vote, voter) on C

all together or none of them ↑

All of these operations SHOULD be done together, as a single ATOMIC COMMIT. However A, B, C could fail during the Transaction... we need a protocol to ensure the atomi commit.

PROTOCOL:



NO UPPER BOUND TO DELIVER MESSAGES ↑

IF WE SEND A MESSAGE INFINITE TIMES, SOONER OR LATER IT WILL ARRIVE ↑

However, we are considering a REALISTIC system (asynchronous and fair-loss). So, to make it work properly we need 4 different ASSUMPTIONS.

ASSUMPTIONS:

SAFETY AC1: All processes that reach a decision must reach the same one (ABORT or COMMIT)

SAFETY AC2: Once a proces reach a decision it doesn't change

SAFETY AC3: COMMIT can be decided only if all processes votes YES

NOTE: if it was an "if and only if" the problem would not be solvable

LIVENESS AC4: if there are NO FAILURES and everybody votes YES, the decision must be COMMIT

FAILURES: But, again, we have to consider failures:

- VOTE REQUEST DOESN'ARRIVE: We can simply set a TIMEOUT. If nothing arrives then Abort.
- VOTE DOESN'ARRIVE: Also the coordinator can set a TIMEOUT. If nothing arrives then Abort.
- DECISION DOESN'T ARRIVE: In this case we cannot set the TIMEOUT... What if we ABORT and then the COMMIT DECISION arrives? All the other processes will commit and our part will be lost invalidating the system. We could proceed in different ways.
    - COOPERATE: thanks to AC1, we can ask to other processes. If anyone aborted we abort; if anyone committed we commit.
    - WAIT FOREVER: Since the channel is fair, sooner or later, the decision will arrive.
    - IMPLEMENT A DISTRIBUTED LOG: So that, in case of crash, we can reload the log and see what happened until now.

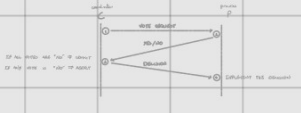
Distributed Log

Unfortunately, we cannot actually create a Distributed Log... It is like an abstraction derived from the collection of the Local Logs of each process. Note that each process (both coordinator or not) must save the local log on its permanent disk, to survive failures. Only by combining the local logs we can reconstruct the whole transaction.

- LOG THEN SEND: Generally, this is the safest approach. If there is a crash the process reads the log, sees that it was supposed to send a message and re-sends it.

- SEND THEN LOG: This is more risky. If there is a crash immediatly after sending a message but before writing the log, after the rebooting the process will have no idea it performed that action.

Let's how this applies to the previous protocol:



- PROCESSES:
  - IF THE VOTE WAS "YES": It's mandatory the Log then Send. Otherwise, with the Send then Log, if the process votes YES, then fails and forget, could resend a different vote ("NO") invalidating the previous message at this point already received by the coordinator

  - IF THE VOTE WAS "NO": Log then Send is better but both are fine. With Send then Log if a process votes "NO" then fails and forget, after the reboot it will abort by default. So the vote doesn't change.

- COORDINATOR: The coordinator crashes, then reboot and read the log:
  1) THERE ARE NO VOTES: probably failed at ①. Re-sends the VOTE REQUEST

  2) THERE IS AT LEAST ONE "NO": Abort directly

3) THERE ARE ALL "YES" BUT SOME VOTE IS MISSING: We could have lost some "no" during the crash (or the vote was "yes" but will never arrive). The coordinato Abort for safety.

4) THERE ARE ALL "YES" BUT NO DECISION LOG: With Log then Send, no decision log mean no decision sent. So we could send COMMIT or ABORT. To stay safe lets assume that we ABORT always in this case. A Send then Log, instead, would be a disaster. Imagine that the coordinator sends "COMMIT" and then crash. After the reboot there is no log, it doesn't know the previous decision and sends "ABORT" for safety. The processes would receive both "COMMIT" and "ABORT"...

5) THERE ARE ALL "YES" AND THE DECISION LOG: In both cases, Log then Send and Send then Log, we simply retransmit the same DECISION as before.

08/10/2026

Limitations

Although the Atomic Commit system works, it has several limitations... First of all, there are the Single Point of Failure and the Unanimity Problem: All processes must vote "YES" to complete the transaction. As a result, if a single process goes down the entire system goes down.

So, we must correct this behaviour. We can do it by introducing the idea of "majority" (rather than unanimity). In this way, if we have 5 processes and 2 of them go down, we can still continue, keeping the system up.

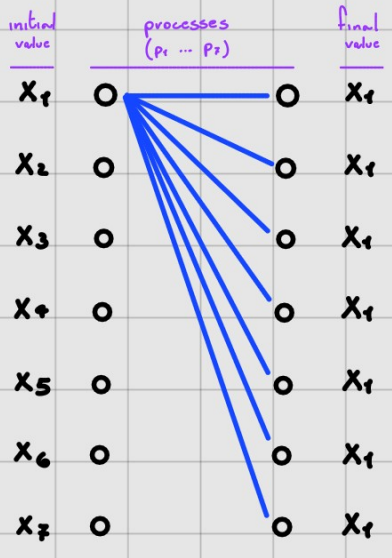
But, as before, we have to manage the CONSENSUS PROBLEM, to guarantee that all processes store the same data (integrity).

The Consensus Problem

Let's assume a distributed system where each process (p_i) has an initial value (x_i). At the end all processes must agree on the same value x, choosen amon $x_1, \dots, x_n$

1) Let's make it easy. Let's assume that the system is SYNCHRONOUS and there are NO FAILURES.

- IDEA: An arbitrary prosses (e.g., p_1) broadcasts its value (x_1) to everybody else. Since there are no failures everybody receives the message and agree on the same value.



2) Let's make it less easy. Lets assume that there could be FAILURES (but the system it's still SYNCHRONOUS).

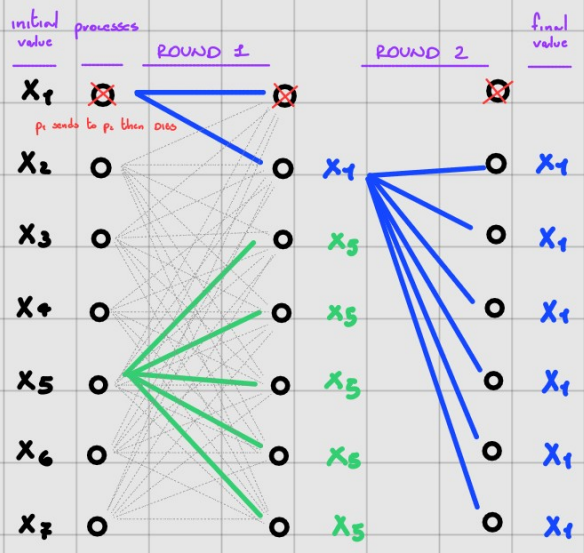
IDEA: Instead of only one process, EACH process broadcasts its value to everybody else. Then, to agree all on a single value, they select only the min value among the received ones.

However, since there are failures we could need to do more ROUNDS to retransmit eventually lost messages.

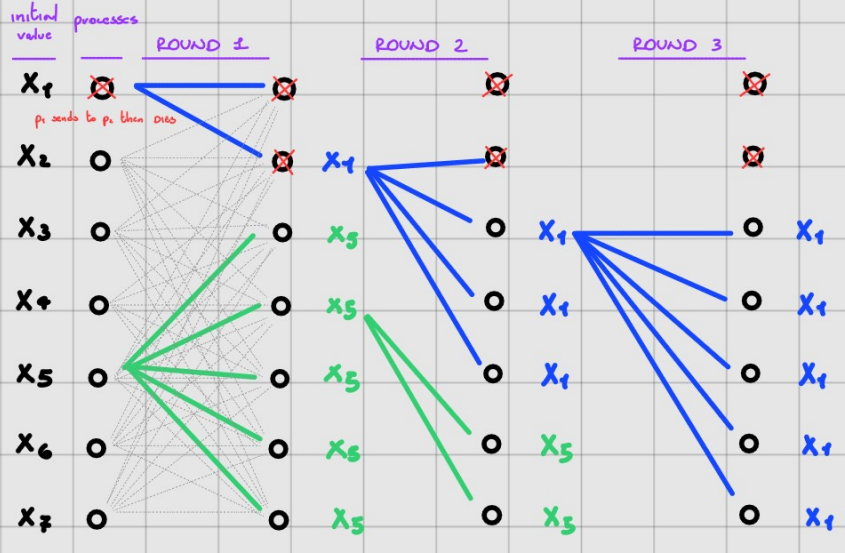
since the system is SYNCHRONOUS we can divide the time in ROUNDS of Δt time

in this case FAIL = CRASH ... so p_4 dies forever after the crash

EXAMPLE: The min value is x_1 BUT p_4 fails after few sends. The second min value is x_5.



So FAILS = 1 ⇒ ROUNDS = 2. What happens with FAILS = 2 ?

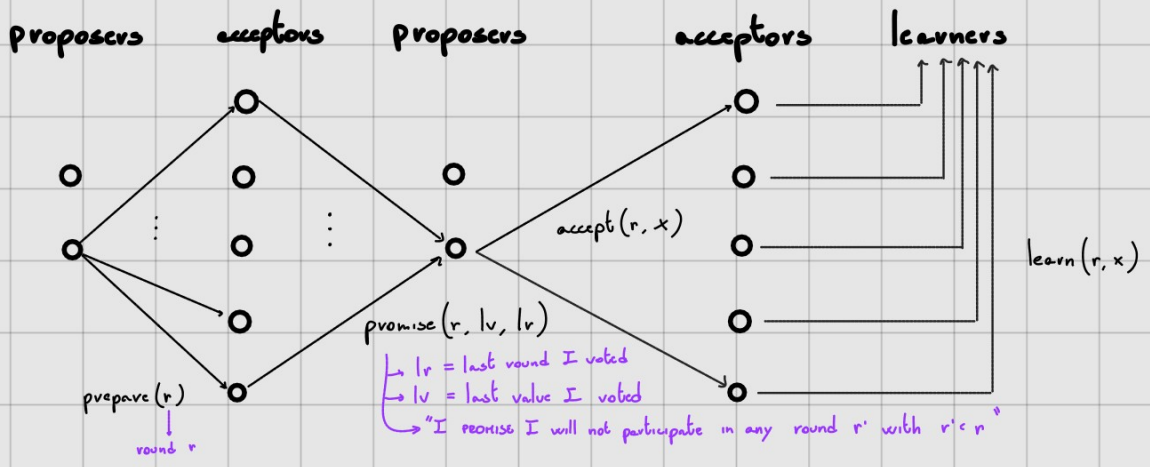


So, in general, we need a minimum of n+1 ROUNDS to solve the problem.

BUT, we are still assuming a SYNCHRONOUS system!
What happens if the system is ASYNCHRONOUS?

This case is much more harder and it needs a new specific well-know protocol.

Paxos



Paxos is a consensus algorithm that operates across 3 different roles: PROPOSER (who suggest values) ACCEPTORS (who actually vote on values) and LEARNERS (who learn/store the choosen value)

ALGORITHM:

1) PREPARE: A proposer initiates a new round by choosing a random number r and sending a promise(r) message to the acceptors

2) PROMISE: When an acceptor receives a prepare(r) message, it replies with a promise(r, lv, lr) message, where:
   - r is the current round
   - lv is the last voted value
   - lr is the last round the acceptor voted in

   In this way the acceptor promises to not partecipate in any round r' < r

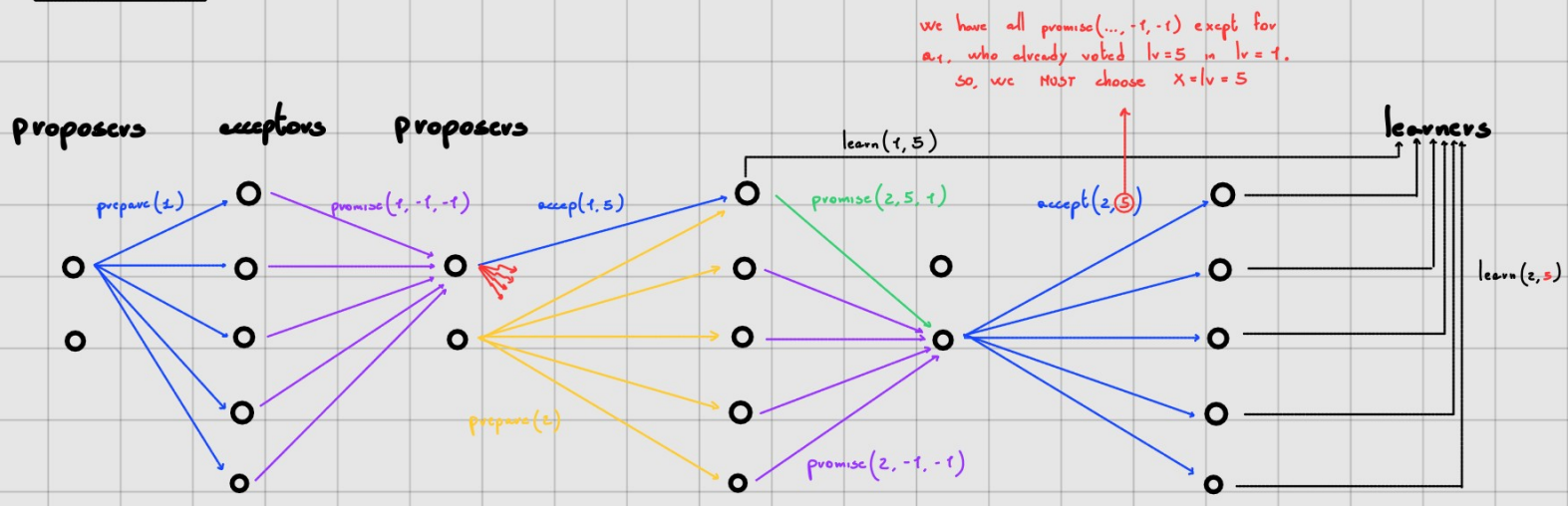
3) ACCEPT: Once the proposer receives promises from a majority of acceptors, it sends an accept(r, x) message to the acceptors, where

$$x = \begin{cases} lv & \text{from the promise with the highest } lr \\ \text{ANY} & \text{if all acceptors have never voted} \end{cases}$$

(lv: LAST VOTED VALUE; lr: THE ROUND lv WAS VOTED IN)

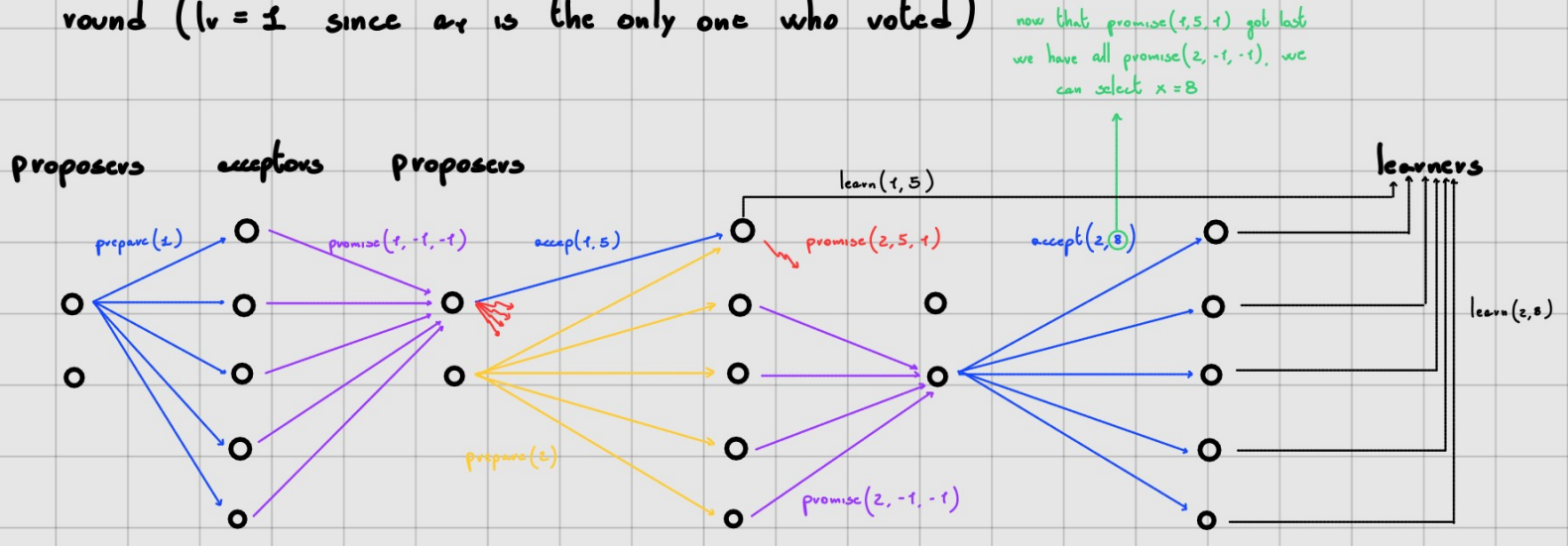
4) LEARN: Once the acceptors receive the accept(r, x) they cast the vote on x sending a learn(r, x) message to the learners, so the system can finalize the decision.

Exercise: a_1 votes for 5 in round 1 and 8 in round 2



WRONG! They all voted 5, not 8.

RIGHT: to ensure that x become equal to 8 the promise(..., 5, ...) MUST be lost! Otherwise, x must be equal to the last voted value (lv = 5) of the max last voted round (lv = 1 since a_1 is the only one who voted)



Consensus Property (CP)

sends a learn(i, x)

def. If acceptor a votes for x in round i, then it is not possible that a different value y was chosen in any of the previous rounds

↳ y gets the majority

DEM: By INDUCTION on i = number of rounds

i = 1 there are no previous rounds ⇒ CP true

i > 1 Assuming that CP is true for all previous rounds $(r_1, \dots, r_{i-1})$. We are currently in round i and we are about to vote x. To reach this voting phase we must have received a promise from a majority quorum (Q). In addition, according to the PAXOS rules, the proposer checks all promises from Q to find the highest last round (lr) any acceptor voted in. Let's call this maximum round J. Clearly, x is the value associated with the round J

- ROUNDS BETWEEN i AND J: The acceptors in Q did not vote in these rounds. If they had voted the maximum round would not have been J. In addition, any other voted value $y \neq x$ could not have been choosen since the remaining acceptors $(A \setminus Q)$ do not form a majority.

- ROUND J AND BEFORE: We know that at least one acceptor from Q voted x during round J. On the previous rounds we apply the inductive hypoteshis, so it is impossible that any diffevnt value $y \neq x$ was chosen before round J.

BEFORE ROUND J	ROUND J	ROUNDS BETWEEN J AND i	ROUND i
Since someone voted x in round J we can apply the INDUCTIVE HYPOTESHIS	At least one acceptor voted x in this round	the acceptors in Q did not vote in these rounds. If they had voted the maximum round would not have been J The other acceptor $(A \setminus Q)$ do not form any majority. So, they cannot choose any $y \neq x$	we are about to vote x Q = quorum = {acceptors who voted x} J = highest lr from the promises of the acceptors in Q